

PSET 2 - PIPELINE DE DATOS END-TO-END

Predicción de fallas de SSD a partir de telemetría SMART (Alibaba Tianchi, 2018-2019)

Gustavo Perla [00325784], Marco Guamán[00336468],
Francisco Terán[00337202], Santiago Arellano[00328370]

Objetivo del proyecto: predecir si un SSD del modelo MC1 fallará en los próximos 30 días a partir de su telemetría SMART diaria. Este pipeline deja los datos limpios, modelados y listos para entrenar ese clasificador.

1. Arquitectura

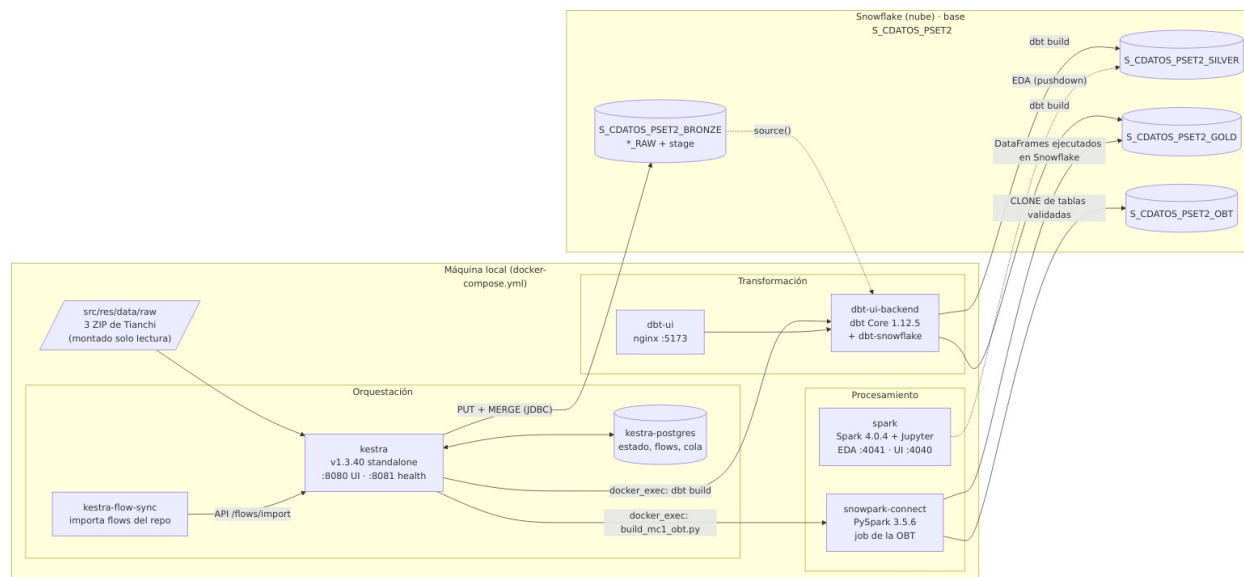


Figura 1. Infraestructura: servicios locales (Docker) y esquemas por capa en Snowflake.

(Figura 2: ver Anexo.)

Snowflake (base S_CDATOS_PSET2) es el destino de todas las capas, con un esquema por capa: `_BRONZE`, `_SILVER`, `_GOLD` y `_OBT`. La infraestructura local está en `docker-compose.yml`:

- kestra (v1.3.40) + kestra-postgres: ingesta y orquestación. Postgres conserva los flows y el historial de ejecuciones entre reinicios.
- Los flows se importan manualmente en Kestra desde los archivos YAML versionados.
- dbt-ui-backend (dbt Core 1.12.5 + dbt-snowflake) y dbt-ui: construyen Silver y Gold.
- spark (Spark 4.0.4 + Jupyter): EDA sobre Silver, con pushdown a Snowflake.
- snowpark-connect (PySpark 3.5.6): job de la OBT. El código es Spark, pero el plan se ejecuta dentro del warehouse de Snowflake, porque unos 115 millones de filas MC1 no caben en un contenedor local.

Flujo de datos: los ZIP de Tianchi se dejan en una landing zone montada como solo lectura. Kestra valida cada CSV, lo sube a un stage interno y lo integra con MERGE en Bronze. dbt lee Bronze con `source()`, tipa y limpia en Silver, y construye el star schema en Gold con `ref()`. Spark lee Gold, calcula features y etiquetas y publica la OBT. Kestra orquesta la ingesta hasta Bronze. dbt y Spark corren en sus propios contenedores y se ejecutan por separado, siguiendo el procedimiento documentado en el repositorio.

Decisiones:

- ELT y no ETL: Bronze conserva el dato original y cualquier regla se puede reprocesar sin volver a descargar la fuente.
- Las credenciales nunca están en el repositorio: se leen de un `.env` ignorado por git. Kestra usa `secret()`, dbt usa `env_var()` y Spark lee variables de entorno.
- Las versiones de las imágenes y paquetes están fijadas para que el entorno sea reproducible.

2. Ingesta

(Figura 3: ver Anexo.)

Fuente: tres ZIP de Tianchi.

Archivo	Contenido
smartlog2018ssd.zip	un CSV por día, 105 columnas, 135.843.663 filas
smartlog2019ssd.zip	un CSV por día, 105 columnas, 137.268.621 filas
ssd_failure_label.csv.zip	un CSV de eventos de falla, 16.305 filas

Cada fila SMART tiene disk_id, ds (fecha), model y 51 atributos SMART, cada uno en versión raw (r_X) y normalizada (n_X).

Granularidad: en la fuente, una fila es un disco en un día; en Bronze, una fila es una fila de un CSV, con clave natural (SOURCE_ARCHIVE, SOURCE_FILE, SOURCE_ROW).

Proceso de carga (flow bronze_ingestion):

1. DDL idempotente: crea, si no existen, el file format, el stage y las tres tablas RAW.
2. Validación antes de cargar (prepare_bronze.py). Exige la cabecera exacta de 105 columnas, el mismo número de campos en cada fila, UTF-8 válido y fechas en los nombres de archivo, e ignora la basura de macOS dentro del ZIP. Cualquier anomalía detiene la carga.
3. Partición: los días se agrupan en paquetes de 2 meses, con hasta 6 en paralelo (un año completo). Dentro de cada paquete se procesa un día a la vez: extraer, hacer PUT al stage y borrar el archivo local. Así nunca se materializan en disco los ~38 GiB de un año.
4. Un único MERGE construye el VARIANT (todo como texto) y el SHA-256 de cada fila.
5. Reconciliación: si una fila del stage no llegó a Bronze, la ejecución termina en FAILED.
6. Limpieza: después de una subida exitosa, se elimina el archivo CSV temporal local.

Por qué VARIANT con texto: si se tipara al cargar, un valor inválido rompería la carga o se perdería. Bronze queda como copia fiel de la fuente y el tipado se hace en Silver.

Trigger y frecuencia:

- bronze_ingestion se inicia manualmente, una vez por dataset o rango histórico. Los CSV de la fuente tienen frecuencia diaria; la carga se ejecuta bajo demanda sobre los ZIP disponibles.
- Silver y Gold se construyen con dbt build; después se ejecuta el job de la OBT en snowpark-connect. No hay un calendario automático ni un flow que encadene las tres etapas.
- concurrency limit 1: nunca hay dos cargas sobre las mismas tablas.
- La operación requiere dejar los ZIP en la landing zone, importar el flow e iniciar la ingesta en Kestra; dbt y Spark se ejecutan por separado. La fuente requiere autenticación para descargar los archivos históricos.

Errores y retries:

- Errores transitorios de Snowflake (red, timeout, warehouse): retry exponencial en todas las tareas Snowflake. DDL: 4 intentos, de 10 s a 1 min. PUT: 4 intentos, hasta 30 min. MERGE: 3 intentos, hasta 2 h. Con warningOnRetry, la ejecución queda en WARNING si necesitó reintentos.
- Errores deterministas de la fuente (falta el ZIP, cambió el esquema, fila corrupta): no se reintentan, porque darían el mismo error.
- dbt build ejecuta los modelos y sus tests. La ejecución posterior de la OBT es independiente: el operador debe comprobar el resultado de dbt antes de iniciarla.

Backfill:

- El backfill se realiza por rango histórico en una ejecución de bronze_ingestion, mediante los inputs requested_start_date y requested_end_date.

- Las fechas de inicio y fin son inclusivas; sin filtros se procesan todos los días disponibles del dataset seleccionado.
- En ambos casos el MERGE es idempotente: una fila nueva se inserta; una igual (mismo hash) no se toca, así que recargar no duplica; una con contenido distinto se actualiza. Nunca se borra nada de Bronze.

3. Data Quality

La calidad se analizó antes de transformar, con notebooks de EDA sobre Silver base (consultas agregadas ejecutadas en Snowflake) y un libro reproducible de análisis de nulos. Las cifras salen de los CSV de evidencia versionados.

Problema (dimensión)	Evidencia	Acción	Justificación
1. Atributos sin ningún valor (completitud)	17 atributos (34 columnas) 100% nulos en 2018 y en 2019.	se eliminan de las tablas procesadas; siguen en Bronze y en Silver base.	sin observaciones no hay base para imputar ni variación que aporte.
2. Atributo presente solo en un año (consistencia)	SMART 211 es 100% nulo en 2018; en MC1 2019 tiene 2.738 valores (99,9955% nulo).	se conserva.	borrarlo por la ausencia en un año sería decidir sin evidencia; lo decidirá la selección de variables.
3. Filas sin ninguna medición (completitud)	490.285 filas en 2018 (0,36%) y 635 en 2019 (0,0005%) con las 102 columnas nulas.	cuarentena en tablas INT_AUDIT_*, con el motivo; no se borran.	no aportan señal, pero se conservan para trazarlas.
4. Nulos parciales (completitud)	4.705 (2018) y 3.379 (2019) combinaciones exactas de nulos. El 97,2% y el 98,1% de las filas tienen 60, 62, 64 o 66 nulos. El patrón dominante cubre al menos el 99,4% de cada modelo, salvo MA1 (72,8%).	no se imputa (ni 0, ni media, ni mediana).	el nulo es estructural ("este modelo no reporta el atributo"); imputar fabricaría señal falsa, y 0 puede ser un valor SMART legítimo.
5. Atributos que MC1 nunca reporta (completitud de la población objetivo)	12 atributos 100% nulos en MC1 en ambos años.	se quitan solo del esquema MC1, que queda con 22 atributos (44 columnas).	no tienen información para MC1; otros modelos sí los usan.
6. disk_id repetido entre modelos (unicidad y consistencia)	115.017 disk_id en 2018 y 113.569 en 2019 aparecen con más de un modelo.	la clave de negocio es (disk_id, model_code) y la clave sustituta es su SHA-256.	usar disk_id solo mezclaría discos físicos distintos.
7. Duplicados (unicidad)	0 duplicados por (disk_id, model, fecha) y 0 duplicados exactos de telemetría en ambos años.	no se deduplica; la unicidad se impone con tests en dbt y validaciones en Spark.	un duplicado futuro rompe el build en vez de esconderse.
8. Pares raw/normalizado (consistencia)	0 patrones con r_X nulo y n_X presente, o al revés.	test smart_pair_missingness_matches en Gold.	es una regla real del dato.
9. Tipos (validez)	todo llega como texto.	TRY_TO_DECIMAL(38,6), TRY_TO_NUMBER y TRY_TO_DATE('YYYYMMDD'), más tests not_null en las claves.	un valor inválido pasa a NULL sin romper la carga y el original queda en Bronze.
10. Fechas (validez)	regla de que la fecha de la fila sea igual a la del archivo, esté en el año del archivo y que las fallas caigan en 2018-2019.	tests singulares en dbt.	detectan errores de parseo o archivos mal nombrados.
11. Etiquetas (completitud)	0 nulos en las 16.305 filas (1.964 fallas de MC1 en 2018 y 8.546 en 2019).	tests not_null.	las etiquetas definen el target.
12. Cambio de prevalencia (consistencia temporal)	la tasa de falla de MC1 pasa de 0,105% (2018) a 0,404% (2019). El patrón dominante de nulos tiene la misma tasa que esa línea base.	no es limpieza; obliga a validar el modelo con split temporal.	un split aleatorio mezclaría dos prevalencias. El patrón dominante de nulos tiene una tasa de falla similar a la base; por ahora no se incorporan features de patrón de nulos.

Precisión: no se dispone de una referencia independiente para verificar la exactitud física de las mediciones SMART. Se conservan los valores originales en Bronze para trazabilidad; el tipado no certifica su exactitud.

La limpieza no pierde filas. Los tests comparan con MINUS en ambos sentidos que limpio + auditoría = Silver base, que la unión de años = 2018 + 2019, y que cada hecho Gold tiene el mismo número de filas que su fuente Silver.

4. Transformaciones y modelado

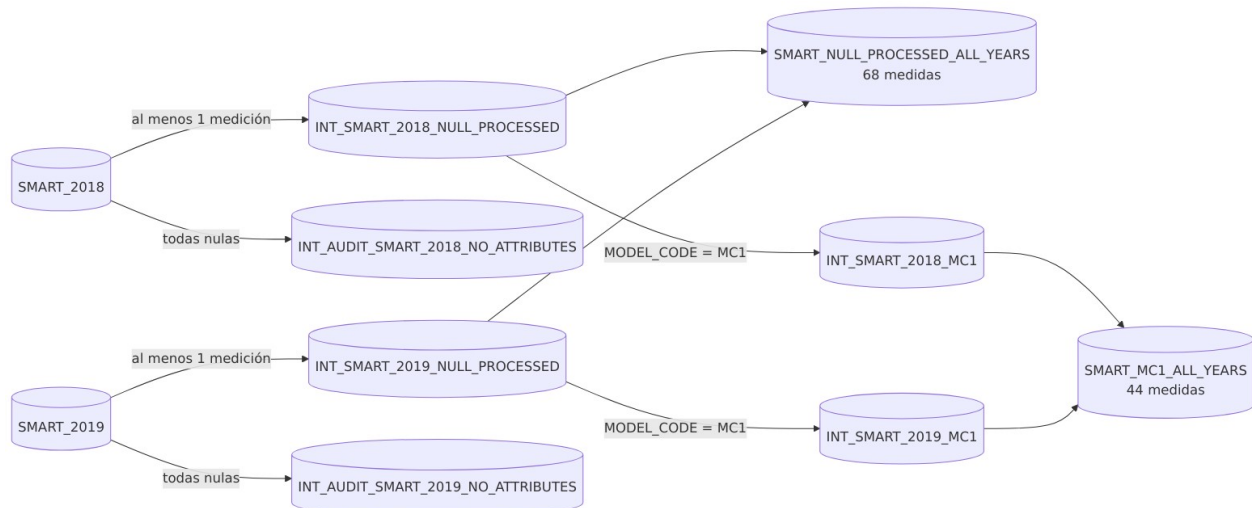


Figura 4. Procesamiento de Silver: limpieza, cuarentena y subconjunto MC1.

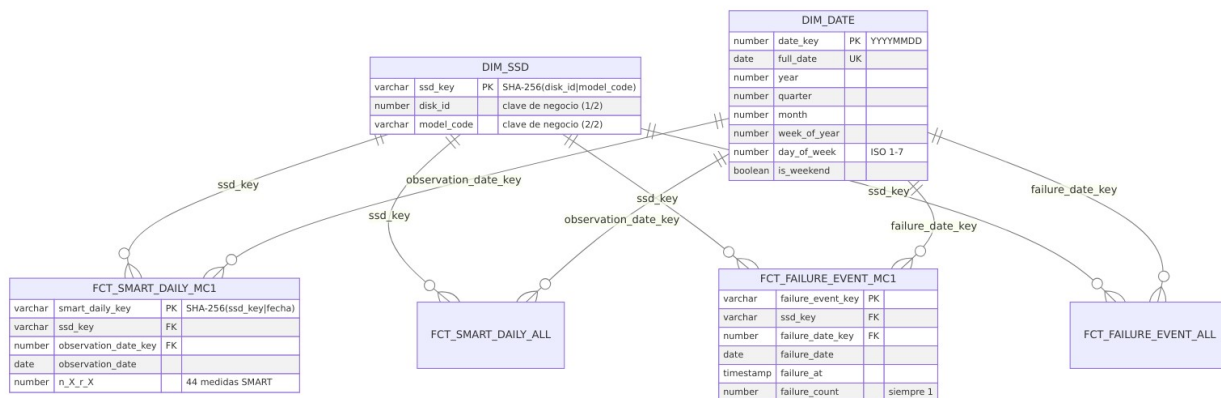


Figura 5. Star schema de Gold (dimensiones conformadas y cuatro hechos).

Bronze (Kestra): tablas *_RAW con el dato original en un VARIANT y su linaje.

Silver (dbt):

- Base (SMART_2018, SMART_2019, SSD_FAILURE_LABELS): materialización incremental con MERGE por silver_record_id = SHA-256(archivo|CSV|fila). Tipa los 105 campos sin filtrar filas. Solo procesa lo que llegó a Bronze después de la última corrida.
- Procesada: INT_*_NULL_PROCESSED (68 medidas), INT_AUDIT_* (cuarentena), INT_*_MC1 (44 medidas) y las tablas finales SMART_NULL_PROCESSED_ALL_YEARS y SMART_MC1_ALL_YEARS, que unen los dos años con UNION ALL. Los años se procesan por separado hasta tener el mismo esquema, para auditarlos por separado.
- Las listas de atributos que se eliminan están definidas una sola vez, en macros, y las usan tanto los modelos como los tests.

Gold (dbt), star schema:

- DIM_SSD: un SSD físico (disk_id, model_code); ssd_key = SHA-256. Se alimenta de las observaciones y de las etiquetas, para incluir discos que fallaron sin telemetría.

- DIM_DATE: un día calendario, continuo y sin huecos (test continuous_date_spine).
- FCT_SMART_DAILY_ALL y FCT_SMART_DAILY_MC1: grain = un SSD en un día de observación; 68 y 44 medidas SMART.
- FCT_FAILURE_EVENT_ALL y FCT_FAILURE_EVENT_MC1: grain = una falla confirmada; medida failure_count = 1. Van separados del hecho SMART porque una falla puede ocurrir un día sin telemetría.
- Dimensiones conformadas: el análisis puede pasar de todos los modelos a MC1 sin cambiar las claves. Las claves con hash son deterministas, así que reconstruir Gold produce las mismas claves.

La ejecución registrada de dbt completó 15 modelos y pasó los 127 tests, sin errores, advertencias ni tests omitidos (PASS=142, WARN=0, ERROR=0, SKIP=0). Los tests validan reglas reales: unique/not_null en las claves; unicidad compuesta del grain (ssd_key, fecha); relationships de cada FK con su dimensión; conteo Gold = conteo Silver; calendario continuo; r_X y n_X nulos a la vez; los hechos MC1 solo contienen MC1; fechas válidas; partición sin pérdida entre limpio y auditoría; columnas eliminadas realmente ausentes. Un SSD con dos fallas el mismo día genera advertencia, no error.

5. Spark y OBT

(Figura 6: ver Anexo.)

Construcción: build_mc1_obt.py usa la API de DataFrames de PySpark con Snowpark Connect. Lee FCT_SMART_DAILY_MC1, FCT_FAILURE_EVENT_MC1, DIM_SSD y DIM_DATE.

Grain: una fila = un SSD MC1 en un día de observación (el mismo de FCT_SMART_DAILY_MC1).

Contenido (847 columnas en la variante RN):

- Identidad (6): claves, DISK_ID y MODEL_CODE.
- Features: para cada uno de los 22 atributos y cada representación (R, N): el valor del día, la media y el conteo en ventanas de 7, 14 y 30 días, el conteo de la ventana anterior, la tendencia (media actual - media anterior) y banderas de validez. Son 19 columnas por atributo. Las ventanas son por rango de días, no de filas, e ignoran los NULL (no imputan).
- Etiqueta (5): TARGET_30D = 1 si la primera falla ocurre entre 1 y 30 días después; 0 si el disco sigue observado 30 días o más sin fallar. CENSORED (sin futuro suficiente), SAME_DAY_FAILURE y POST_FAILURE quedan con TARGET nulo, para no etiquetar como sano un disco del que simplemente faltan datos.
- Se publican 3 variantes: RN (847 columnas), R y N (429 cada una), para comparar representaciones en el modelado sin recalcular ventanas.

Validación de joins y del número de observaciones:

1. Grain de entrada: filas = claves distintas = pares (SSD, fecha) distintos.
2. Las dimensiones no tienen claves repetidas (si las tuvieran, el join multiplicaría filas) y no hay hechos huérfanos (left_anti).
3. Conteo después del join con las dimensiones igual al conteo antes. Es la prueba directa de que no se duplicaron ni perdieron observaciones.
4. Sobre la tabla ya escrita: mismo número de filas que Gold, sin duplicados, ventanas y etiquetas recalculadas iguales a las guardadas, solo MC1.
5. Las variantes R y N tienen exactamente las mismas filas que RN (exceptAll en ambos sentidos).

Si falla una validación, no se inicia la publicación. Las tablas stage validadas se publican secuencialmente como RN, R y N mediante CREATE OR REPLACE ... CLONE. Cada reemplazo es individual; una falla durante la publicación puede dejar variantes de distintas corridas. El resultado y las métricas de la corrida se registran en OBT_MC1_RUN_AUDIT.

Star schema frente a OBT en este proyecto:

- Star schema: análisis y reportes (fallas por mes, comparación entre modelos, tasas por año). Las dimensiones conformadas permiten agregar por cualquier eje sin duplicar datos.
- OBT: entrenamiento y scoring del clasificador. Necesita una fila por observación con todas las features y la etiqueta, sin joins ni ventanas en el momento de entrenar. Es un derivado que se reconstruye desde Gold.

6. Batch vs. streaming

Batch es suficiente:

- Frecuencia de los datos: SMART se publica como un archivo diario, así que la unidad natural ya es un lote. La fuente es además histórica.
- Latencia requerida: la pregunta es si el disco fallará en los próximos 30 días, y la acción (migrar datos, programar el reemplazo) se planifica en horas o días. Un resultado diario llega con semanas de margen; bajar a segundos no cambia ninguna decisión.
- Features: las ventanas de 7, 14 y 30 días se calculan una vez que el día está completo.
- Calidad y costo: cada lote se valida completo (esquema, reconciliación, tests) antes de pasar a la siguiente capa. Streaming añadiría un broker, estado por disco y manejo de datos tardíos sin ningún beneficio.

Se justificaría streaming si:

- el horizonte fuera de minutos u horas (desconectar un disco antes de que corrompa un volumen);
- la predicción disparara una acción automática inmediata (sacar el disco de un RAID);
- los discos emitieran SMART de forma continua a un broker, en vez de un CSV diario;
- hubiera señales de alto valor que pierden utilidad si se esperan horas (saltos de sectores reasignados o de errores CRC).

En ese caso cambiarían la ingesta (un broker en lugar de ZIP), las features (ventanas con estado en streaming) y el serving del modelo (en línea). El star schema y la OBT seguirían existiendo para el entrenamiento histórico.

7. Limitaciones

Datos:

- La relación numérica entre r_X y n_X no se puede reconstruir con los exportes agregados; por eso se publican las tres variantes de la OBT.
- SMART 211 es casi totalmente nulo en MC1; su utilidad se decidirá en la selección de variables.
- La etiqueta usa la primera falla de cada SSD.
- Los datos son observacionales: no permiten afirmar causalidad (firmware, edad y carga de trabajo son confusores).

Arquitectura:

- La descarga de los ZIP es manual, porque Tianchi no tiene API.
- El socket de Docker está montado en Kestra, pero el flow versionado no lo utiliza para invocar dbt ni Spark. La orquestación automática Bronze → OBT queda como limitación de la arquitectura.
- El incremental de Silver no es CDC completo: si cambia la lógica de parseo hay que correr dbt con --full-refresh.
- La lista de atributos SMART aparece en varios lugares (ingesta, macros de dbt y job de Spark), así que agregar uno exige tocar varios archivos.
- Gold y Silver procesada se reconstruyen completos en cada dbt build.
- La generación de las capas se realiza por etapas. El repositorio no incluye un flow único de orquestación Bronze → OBT; su automatización queda pendiente.

Anexo

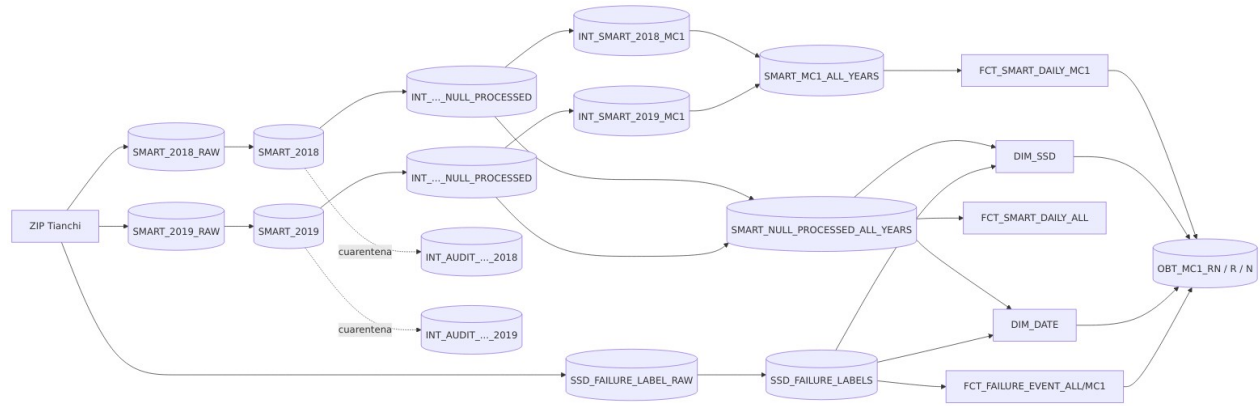


Figura 2. Linaje de datos de Bronze a la OBT.

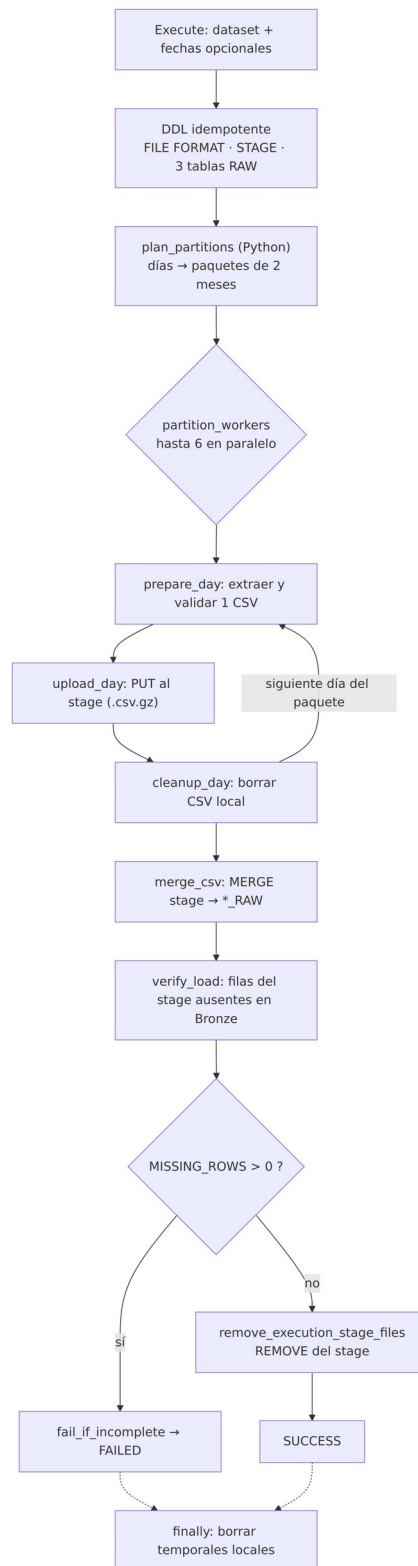


Figura 3. Flow de ingesta bronze_ingestion en Kestra.

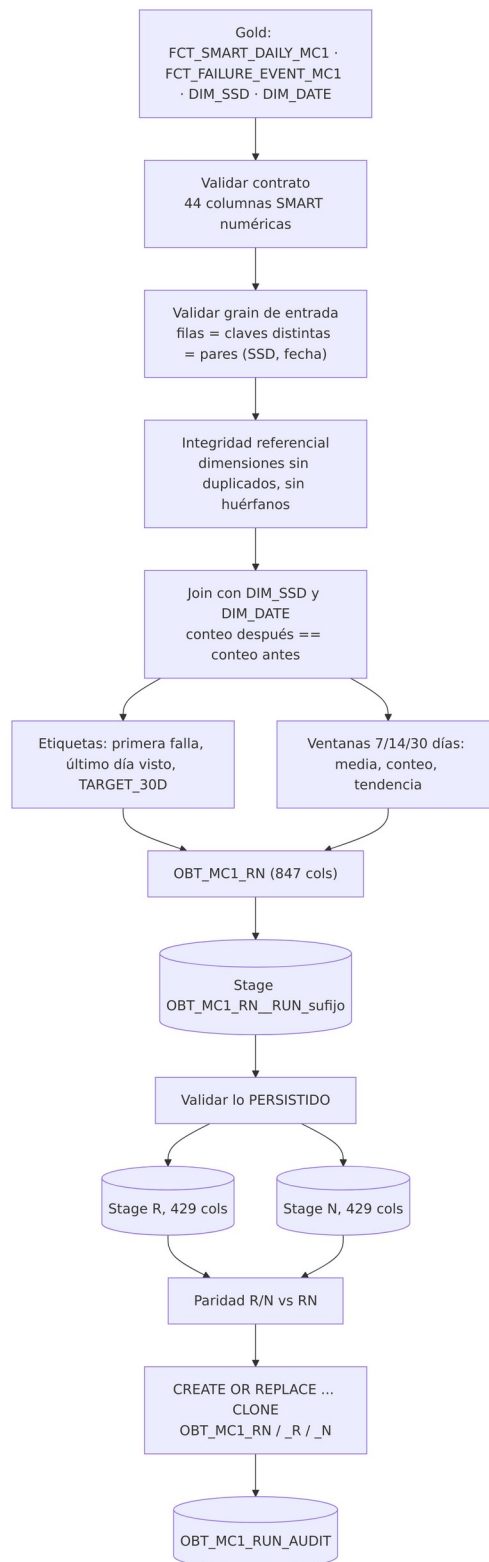


Figura 6. Construcción, validación y publicación de la OBT.

el repositorio (src/main/psets/pset_2) contiene docker-compose.yml, src/res/env/.env.example, README.md, los flows de Kestra, el proyecto dbt, el job de Spark y la documentación detallada en docs/ (incluida la ejecución paso a paso en docs/08_ejecucion.md).